

10. Pandoc Single Markdown & PDF Compilation

Architecture, Manifest Ecosystem, Multi-Language Implementations, and Diagnostic Tooling

Mars Rover Design Team

October 05, 2026

Contents

1	Chapter 01: Protocol Specification & Wire Format	4
1.1	1. Design Philosophy & Requirements	4
1.2	2. Wire Format & Header Layout	4
1.2.1	Byte-by-Byte Header Field Breakdown	4
1.2.2	Maximum Packet Limits	5
1.3	3. Endianness & Network Byte Ordering	5
1.4	4. Supported Data Types Reference	6
1.5	5. System Reserved Packets (Control IDs 1-6)	7
1.6	6. Network IP & Subnet Topology	8
1.6.1	Standard Port Allocations	8
1.6.2	Static Subnet Allocations	8
1.7	7. Packet Validation Rules	8
2	Chapter 02: Transport Layers: UDP vs TCP	9
2.1	1. Architectural Decision Matrix	9
2.2	2. UDP Transport & The Subscription Model	9
2.2.1	The UDP Subscription Lifecycle	9
2.2.2	Subscription Protocol Implementation Details	10
2.3	3. TCP Transport & Stream Framing	10
2.3.1	Handling TCP Stream Delimitation	10
2.3.2	TCP Frame Extraction Algorithm	10
2.4	4. Connection State Machine & Fault Recovery	11
2.4.1	Link Monitoring & Keep-Alives	11
3	Chapter 03: The Manifest Ecosystem & Automated CI Pipeline	12
3.1	1. Anatomy of <code>manifest.json</code>	12
3.1.1	Board Declaration Schema	12
3.1.2	Comment Formatting Conventions	13
3.2	2. Automated README Generation	14
3.3	3. The Webhook Dispatch Pipeline	14
3.3.1	End-to-End Synchronization Architecture	15
3.4	4. Downstream Code Binding Generators	15
3.4.1	C++ Code Generation (<code>RoveComm_CPP</code>)	15
3.4.2	C# Code Generation (<code>RoveComm_CSharp</code>)	16
3.5	5. How to Add a New Packet to RoveComm (Tutorial)	16
4	Chapter 04: C++ Implementation Guide	18
4.1	1. Architecture & Threading Model	18
4.1.1	Platform Compatibility	18

4.2	2. Core Classes & Data Structures	18
4.2.1	rovecomm::RoveCommPacket<T>	18
4.2.2	rovecomm::RoveCommUDP	18
4.2.3	rovecomm::RoveCommTCP	19
4.3	3. Production Code Examples	19
4.3.1	Example 1: Transmitting Drive Commands over UDP	19
4.3.2	Example 2: Subscribing to GPS Telemetry with Asynchronous Callbacks	20
4.3.3	Example 3: Transmitting Critical E-Stop Command over TCP	21
4.4	4. CMake Integration in Downstream Projects	21
5	Chapter 05: C# (.NET / Blazor) Implementation Guide	22
5.1	1. Architecture & BaseStation Integration	22
5.2	2. Core API Reference	22
5.2.1	RoveCommPacket<T>	22
5.2.2	RoveCommService	22
5.3	3. Production Code Examples	23
5.3.1	Example 1: Registering RoveCommService in Program.cs	23
5.3.2	Example 2: Subscribing to Telemetry in a Blazor Component	23
5.3.3	Example 3: Dispatching Drive Commands from Joystick Input	24
6	Chapter 06: Python Implementation Guide	25
6.1	1. Architecture of rovecomm.py	25
6.1.1	Struct Packing & Header Format	25
6.2	2. Core Python Classes	25
6.2.1	RoveCommPacket	25
6.2.2	RoveComm Node	25
6.3	3. Production Code Examples	26
6.3.1	Example 1: Publishing GPS Doppler Velocity from Python	26
6.3.2	Example 2: Subscribing & Logging Rover Motor Currents	26
7	Chapter 07: Embedded (Arduino / Microcontroller) Guide	28
7.1	1. Embedded Constraints & Architecture	28
7.2	2. Production Firmware Example	28
8	Chapter 08: RoveComm Tester Software & Diagnostic Tooling	31
8.1	1. Overview & System Architecture	31
8.2	2. Installation & Quick Start	31
8.2.1	Prerequisites	31
8.2.2	Setup Instructions	31
8.3	3. Core Modules Deep Dive	32
8.3.1	A. The Packet Sender (QtSender.py)	32
8.3.2	B. The Packet Sniffer & Receiver (QtReciever.py)	32
8.3.3	C. Gamepad Teleoperation (XboxController.py)	32
8.4	4. Common Diagnostic Workflows	32
8.4.1	Diagnostic 1: Verifying Board Network Liveness (Ping)	32
8.4.2	Diagnostic 2: Simulating Sensor Telemetry for Autonomy Testing	33
9	Chapter 09: RoveSoDocs Hub Integration Playbook	34
9.1	1. Dual-Docset Subpath Convention	34
9.2	2. The 6-Step Integration Checklist	34
9.2.1	Step 1: Duplicate Jekyll Section in RoveSoDocs/.github/workflows/deploy.yml	34
9.2.2	Step 2: Merge Artifact in assemble Job	35
9.2.3	Step 3: Register SPA Fallback in NotFoundContent.vue	35
9.2.4	Step 4: Index Guide in MiniSearch Search Engine	36
9.2.5	Step 5: Add Feature Card on Homepage (index.md)	36

9.2.6	Step 6: Maintain Visual Grid Balance	36
10	Chapter 10: Pandoc Single Markdown & PDF Compilation	37
10.1	1. Toolchain Prerequisites	37
10.1.1	Ubuntu / Debian / WSL2	37
10.1.2	Windows (Native)	37
10.2	2. Compilation Script (<code>compile_rovecomm_pandoc.sh</code>)	37
10.2.1	Script Execution Pipeline	37
10.3	3. Formatting Rules for Pandoc Compatibility	37

1 Chapter 01: Protocol Specification & Wire Format

The **RoveComm** protocol is the custom application-layer binary messaging protocol engineered by the Missouri S&T Mars Rover Design Team (MRDT). It interconnects all computing nodes on the rover and ground control station – including embedded microcontrollers (Teensy 4.1, STM32, SAM), single-board computers (NVIDIA Jetson AGX Orin, Raspberry Pi 5), high-level autonomy software stacks, simulation engines (Unreal Engine 5), and the mission control web dashboard (Blazor BaseStation).

This chapter defines the low-level wire format, byte framing, data types, system-reserved packets, and network IP routing topology for **RoveComm Version 3**.

1.1 1. Design Philosophy & Requirements

Traditional general-purpose messaging protocols (such as ROS 2 DDS, Protobuf over gRPC, or JSON over WebSockets) introduce substantial serialization overhead, large memory footprints, and complex runtime dependencies that cannot operate efficiently on resource-constrained embedded microcontrollers.

RoveComm is engineered around four core tenets:

1. **Minimal Header Overhead:** A fixed 6-byte header allows embedded microcontrollers to parse packets with zero dynamic memory allocation and minimal CPU cycles.
2. **Deterministic Serialization:** Strict big-endian network byte ordering guarantees binary interoperability across heterogeneous CPU architectures (x86_64, ARM Cortex-A78AE, ARM Cortex-M7, and RISC-V).
3. **Schema-Driven Network Contract:** The entire team’s networking catalog – every board, IP address, command, telemetry stream, error flag, and enum – is declared in a single central repository (`RoveComm_Base/manifest.json`), eliminating packet definition drift across multidisciplinary subteams.
4. **Dual Transport Support:** Native abstraction over both low-latency connectionless UDP (for high-frequency sensor telemetry and teleoperation drive commands) and connection-oriented TCP (for mission-critical state transitions, file transfers, and E-Stop commands).

1.2 2. Wire Format & Header Layout

Every RoveComm packet transmitted over Ethernet or Wi-Fi begins with an identical **6-byte header**, followed by an optional variable-length payload:

0	1	2	3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1			
+++++			
RoveComm Ver	Data ID	Data Count	
+++++			
(Cnt cont'd)	Data Type	Payload Data ...	
+++++			
	...		
+++++			

1.2.1 Byte-by-Byte Header Field Breakdown

Byte Offset	Field Name	Data Type	Description
Byte 0	RoveComm Version	uint8_t	Protocol specification version. For current MRDT rovers, this value MUST equal 3 (0x03). Packets with mismatching versions are rejected.
Bytes 1-2	Data ID	uint16_t	Unique identifier (0-65535) denoting the command, telemetry stream, or error type as registered in manifest.json . Big-endian network byte order.
Bytes 3-4	Data Count	uint16_t	Number of elements of type Data Type contained in the following payload. Big-endian network byte order.
Byte 5	Data Type	uint8_t	Primitive data type enum (values 0 to 8) defining how the payload bytes must be decoded.
Bytes 6+	Data Payload	Variable	The binary array of data elements. Total payload byte size is calculated as $\text{DataCount} \times \text{sizeof}(\text{DataType})$.

1.2.2 Maximum Packet Limits

- **Header Size:** Exactly 6 bytes (`ROVECOMM_PACKET_HEADER_SIZE = 6`).
- **Maximum Data Count:** 65,535 elements (`ROVECOMM_PACKET_MAX_DATA_COUNT = 65535`).
- **Standard Maximum Transmission Unit (MTU):** For standard Ethernet UDP datagrams without IP fragmentation, payloads are typically kept under 1,466 bytes (1500 – 20 (IP) – 8 (UDP) – 6 (RoveComm)). Large telemetry bursts (such as Raman CCD spectrometer readings consisting of 2,560 elements) are split into multiple smaller packets (`RamanReading_Part1` through `RamanReading_Part5`) to prevent packet fragmentation.

1.3 3. Endianness & Network Byte Ordering

All multi-byte numeric fields in both the RoveComm header (`DataID`, `DataCount`) and the payload (`int16_t`, `uint16_t`, `int32_t`, `uint32_t`, `float`, `double`) **MUST** be transmitted in **Big-Endian (Network Byte Order)**.

When serializing and deserializing packets on little-endian architectures (such as x86 and ARM):

- **16-bit integers:** Converted using `htons()` (Host TO Network Short) and `ntohs()`.
- **32-bit integers:** Converted using `htonl()` (Host TO Network Long) and `ntohl()`.
- **64-bit integers & doubles:** Converted using 64-bit bit-shift swapping (`htonll()` and `ntohll()`).

- **Single-precision floats (FLOAT_T):** Serialized according to IEEE 754 standard, with the 4 raw bytes swapped via `htonl()` before network transmission.
- **8-bit integers & characters (INT8_T, UINT8_T, CHAR):** Transmitted directly without byte-swapping.

```
// 64-bit Big-Endian Byte Swap Helper
#define htonll(x) ((1 == htonl(1)) ? (x) : (((uint64_t) htonl((x) & 0xFFFFFFFFUL)) << 32) | htonl((uint64_t) (x) >> 32))
#define ntohll(x) ((1 == ntohl(1)) ? (x) : (((uint64_t) ntohl((x) & 0xFFFFFFFFUL)) << 32) | ntohl((uint64_t) (x) >> 32))
```

1.4 4. Supported Data Types Reference

RoveComm defines 9 standard primitive data types mapped to fixed integer identifiers in `manifest.json`:

Enum ID	Type Name	Byte Size	Description	C++ Equivalent	C# (.NET) Type	Python struct Code
0	INT8_T	1	Signed 8-bit integer	<code>int8_t</code>	<code>sbyte</code>	<code>'b'</code>
1	UINT8_T	1	Unsigned 8-bit integer / byte / bitmask	<code>uint8_t</code>	<code>byte</code>	<code>'B'</code>
2	INT16_T	2	Signed 16-bit integer	<code>int16_t</code>	<code>short</code>	<code>'h'</code>
3	UINT16_T	2	Unsigned 16-bit integer	<code>uint16_t</code>	<code>ushort</code>	<code>'H'</code>
4	INT32_T	4	Signed 32-bit integer	<code>int32_t</code>	<code>int</code>	<code>'l'</code>
5	UINT32_T	4	Unsigned 32-bit integer	<code>uint32_t</code>	<code>uint</code>	<code>'L'</code>
6	FLOAT_T	4	IEEE 754 single-precision floating point	<code>float</code>	<code>float</code>	<code>'f'</code>
7	DOUBLE_T	8	IEEE 754 double-precision floating point	<code>double</code>	<code>double</code>	<code>'d'</code>
8	CHAR	1	8-bit ASCII character / null-terminated string	<code>char</code>	<code>char</code>	<code>'c'</code>

1.5 5. System Reserved Packets (Control IDs 1-6)

Data IDs 1 through 6 are globally reserved across all boards for protocol control and connection management. They must not be assigned to board-specific commands or telemetry:

System Data ID	Packet Name	Transport	Description
1	PING	UDP / TCP	Sent to verify socket liveness and network connectivity. The receiving node must respond immediately with a PING_REPLY. Payload is empty (DataCount = 0).
2	PING_REPLY	UDP / TCP	Transmitted in response to a PING packet. Carries round-trip latency verification. Payload is empty (DataCount = 0).
3	SUBSCRIBE	UDP	Instructs the receiving UDP server to add the sender's IP and port to its active telemetry distribution list. Payload is empty (DataCount = 0).
4	UNSUBSCRIBE	UDP	Instructs the receiving UDP server to remove the sender from its active telemetry distribution list. Payload is empty (DataCount = 0).
5	INVALID_VERSION	UDP / TCP	Emitted by a receiving node when an incoming packet's Version field does not match the receiver's supported version. Payload contains the receiver's supported version as UINT8_T.
6	NO_DATA	UDP / TCP	Sent as a heartbeat or acknowledgment indicating zero payload update.

1.6 6. Network IP & Subnet Topology

MRDT operates an isolated physical Gigabit Ethernet and 5 GHz / 900 MHz Ubiquiti wireless network. Computing devices and microcontroller boards use static IP addresses partitioned into dedicated operational subnets:

1.6.1 Standard Port Allocations

- 11000: Default Ethernet UDP Port (`ethernetUDPPort`).
- 12000: Default Ethernet TCP Port (`ethernetTCPPort`).

1.6.2 Static Subnet Allocations

Subnet Range	Network Domain	Description & Primary Devices
192.168.2.0/24	Main Rover Subnet	Core Board (192.168.2.110) Power Management System - PMS (.102) Navigation Board (.104) Multimedia Board (.106) Autonomy Jetson Computer (.108) Robotic Arm Controller (.112)
192.168.3.0/24	Science Subnet	Raman Spectrometer Board (192.168.3.105) Science Carousel Controller (.106)
192.168.100.0/24	Drone Subnet	Drone Flight Controller (192.168.100.102)
192.168.254.0/24	Network Core	Rover Managed Switch (192.168.254.1) BaseStation Managed Switch (192.168.254.2)
10.0.0.0/24	RF Rocket Links	Rover 900 MHz Ubiquiti Rocket (10.0.0.3) BaseStation 900 MHz Rocket (10.0.0.4) Rover 5 GHz Ubiquiti Rocket (10.0.0.19) BaseStation 5 GHz Rocket (10.0.0.20)
127.0.0.1	Local Loopback	RoveSoSimulator (127.0.0.1)

1.7 7. Packet Validation Rules

When receiving a RoveComm frame, implementations **MUST** execute the following verification pipeline before passing data to subsystem callbacks:

1. **Length Check:** Total bytes received must be ≥ 6 bytes.
2. **Version Check:** Byte 0 must equal `ROVECOMM_VERSION` (3). If invalid, emit an `INVALID_VERSION` packet back to the sender and discard.
3. **Payload Bound Check:** The expected payload size ($S = \text{DataCount} \times \text{DataTypeSize}$) must equal $\text{TotalBytesReceived} - 6$. If fewer bytes are present, the frame is corrupted and must be dropped.
4. **Data ID Verification:** Ensure `DataID` is known in the board's manifest before executing actuator commands.

2 Chapter 02: Transport Layers: UDP vs TCP

RoveComm provides unified packet framing over both **User Datagram Protocol (UDP)** and **Transmission Control Protocol (TCP)**. Each transport layer is selected based on the specific operational characteristics of the subsystem: high-frequency streaming sensor telemetry requires minimal latency and can tolerate occasional packet drop, whereas discrete actuator switching, safety-critical E-Stop triggers, and large scientific datasets require guaranteed, ordered delivery.

2.1 1. Architectural Decision Matrix

The following decision matrix governs the transport layer selection across all MRDT subsystems:

Metric	UDP Transport (Port 11000)	TCP Transport (Port 12000)
Connection Model	Connectionless (fire-and-forget datagrams)	Connection-oriented (persistent 3-way handshake stream)
Delivery Guarantee	Best-effort (packets may drop or arrive out of order)	Guaranteed delivery with automated link-layer retransmission
Ordering	No ordering guarantee	Guaranteed strict FIFO packet ordering
Latency Profile	Sub-millisecond latency; zero head-of-line blocking	Latency spikes during packet loss due to ACK timeouts
Header Overhead	Low (8-byte UDP header + 6-byte RoveComm header)	Higher (20-byte TCP header + 6-byte RoveComm header)
Primary Use Cases	- Drive wheel motor speeds- Real-time IMU roll/pitch/yaw- Continuous GPS Doppler velocities- Teleoperation joystick commands- LiDAR telemetry streams	- PMS E-Stop & Suicide triggers- Power bus toggle commands- Raman CCD spectrometer dataset downloads- Autonomous mission waypoint injection- Subsystem configuration updates

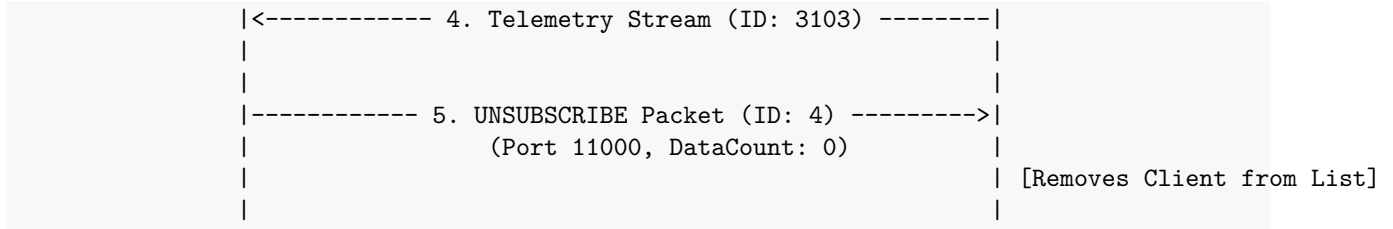
2.2 2. UDP Transport & The Subscription Model

UDP operates without persistent connection overhead. On the rover network, microcontroller boards broadcast high-frequency telemetry (e.g., motor current at 50 Hz, IMU heading at 100 Hz) using the **RoveComm Subscription Model**.

2.2.1 The UDP Subscription Lifecycle

Rather than broadcasting telemetry to all network devices (which degrades wireless bandwidth over 900 MHz Rocket links), RoveComm servers only stream telemetry to clients that have explicitly requested it:

[Client Node (BaseStation / Autonomy)]	[Server Node (Core / PMS / Nav)]
----- 1. SUBSCRIBE Packet (ID: 3) ----->	
(Port 11000, DataCount: 0)	
	[Registers Client IP & Port]
	[Adds to vSubscribers List]
<----- 2. Telemetry Stream (ID: 3100) -----	
<----- 3. Telemetry Stream (ID: 3101) -----	



2.2.2 Subscription Protocol Implementation Details

1. **Subscribing:** To subscribe, a client transmits a UDP packet to the target board's IP address on port 11000 with `DataID = 3` (SUBSCRIBE), `DataType = UINT8_T`, and `DataCount = 0`.
2. **Subscriber Storage:** The receiving board records the sender's IP address and UDP port in an internal table. In `RoveComm_CPP`, this is bounded by `ROVECOMM_ETHERNET_UDP_MAX_SUBSCRIBERS = 10`.
3. **Telemetry Streaming:** During each board execution loop (typically 10 ms to 100 ms intervals), the board iterates through its active subscribers and sends copies of its telemetry packets using `unicast sendto()`.
4. **Unsubscribing:** When a client shuts down or changes operational modes, it sends an UNSUBSCRIBE packet (`DataID = 4`) to remove itself from the server's streaming loop.
5. **Heartbeat & Stale Subscriber Pruning:** If a subscriber fails to send any packet or heartbeat for over 10 seconds, embedded firmware automatically evicts the stale subscriber to conserve network resources.

2.3 3. TCP Transport & Stream Framing

TCP provides reliable, ordered data transfer over port 12000. However, because TCP is a byte-stream protocol rather than a message-oriented protocol, multiple RoveComm packets may arrive coalesced within a single TCP segment, or a single packet may be fragmented across multiple TCP reads.

2.3.1 Handling TCP Stream Delimitation

Implementations of RoveComm over TCP **MUST** implement stateful buffering to extract individual packets cleanly:

```

Incoming TCP Stream Buffer:
[ 0x03 | 0x0F 0xA0 | 0x00 0x02 | 0x06 | 0x3F 0x80 0x00 0x00 | 0x3F 0x80 0x00 0x00 ] [ 0x03 ...
|<- - - - - Fixed 6-Byte Header - - - - - ->|<- - - Payload (8 Bytes) - ->|
|<- - - - - Complete RoveComm Packet (14 Bytes) - - - - - ->|

```

2.3.2 TCP Frame Extraction Algorithm

```

void ProcessTCPBuffer(std::vector<uint8_t>& receiveBuffer)
{
    while (receiveBuffer.size() >= ROVECOMM_PACKET_HEADER_SIZE)
    {
        // 1. Inspect Header
        uint8_t version = receiveBuffer[0];
        if (version != ROVECOMM_VERSION)
        {
            // Drop corrupt byte and resynchronize
            receiveBuffer.erase(receiveBuffer.begin());
            continue;
        }
    }
}

```

```

uint16_t dataId = (receiveBuffer[1] << 8) | receiveBuffer[2];
uint16_t dataCount = (receiveBuffer[3] << 8) | receiveBuffer[4];
uint8_t dataType = receiveBuffer[5];
size_t elementSize = GetDataTypeSize(dataType);
size_t totalPacketSize = ROVECOMM_PACKET_HEADER_SIZE + (dataCount * elementSize);

// 2. Check if complete packet has arrived
if (receiveBuffer.size() < totalPacketSize)
{
    // Incomplete packet; wait for next TCP socket read
    break;
}

// 3. Extract and dispatch packet
RoveCommData packetBytes;
std::copy(receiveBuffer.begin(), receiveBuffer.begin() + totalPacketSize, packetBytes.unBytes);
DispatchCallbacks(packetBytes);

// 4. Remove processed frame from buffer
receiveBuffer.erase(receiveBuffer.begin(), receiveBuffer.begin() + totalPacketSize);
}
}

```

2.4 4. Connection State Machine & Fault Recovery

2.4.1 Link Monitoring & Keep-Alives

Wireless connections in the desert (Hanksville, UT for URC) experience RF multipath interference and periodic link drops. The TCP client layer (in Autonomy or BaseStation) implements an automated state machine for fault recovery:

1. **Periodic Ping:** Every 1,000 ms of socket inactivity, the client transmits a PING packet (DataID = 1).
2. **Ping Timeout:** If the server fails to respond with a PING_REPLY (DataID = 2) within 2,500 ms, the connection is marked broken.
3. **Graceful Reconnection:** The client closes the existing socket file descriptor, resets socket buffers, waits for an exponential backoff period (250 ms, 500 ms, 1000 ms, max 3000 ms), and attempts `connect()` until the board is back online.
4. **Actuator Safety Timeout:** All embedded actuator boards (Drive, Arm, Science) feature a 500 ms watchdog. If no valid command packet is received within 500 ms, the board immediately sets motor outputs to zero (EStop safe state).

3 Chapter 03: The Manifest Ecosystem & Automated CI Pipeline

In a large collegiate robotics team with dozens of engineers writing code in C++, C#, Python, and Arduino across separate microcontrollers, Jetson computers, and ground station dashboards, packet definitions inevitably diverge if maintained manually.

To solve this, MRDT utilizes **RoveComm_Base** as the single authoritative source of truth. Every board IP, command, telemetry metric, error code, and enumeration is declared in a single JSON schema (**manifest.json**). Whenever a developer commits an update to **manifest.json**, an automated GitHub Actions CI pipeline regenerates human-readable documentation, emits cross-repository webhooks, updates git submodules across downstream language repositories, and triggers code generators to compile native language bindings (**RoveCommManifest.h**, **RoveCommBoards.cs**, etc.).

3.1 1. Anatomy of manifest.json

The root configuration of **manifest.json** defines protocol-wide network constants, primitive types, and system packets:

```
{
  "ManifestSpecVersion": 3,
  "DataTypes": {
    "INT8_T": 0,
    "UINT8_T": 1,
    "INT16_T": 2,
    "UINT16_T": 3,
    "INT32_T": 4,
    "UINT32_T": 5,
    "FLOAT_T": 6,
    "DOUBLE_T": 7,
    "CHAR": 8
  },
  "dataSizes": [1, 1, 2, 2, 4, 4, 4, 8, 1],
  "SystemPackets": {
    "PING": 1,
    "PING_REPLY": 2,
    "SUBSCRIBE": 3,
    "UNSUBSCRIBE": 4,
    "INVALID_VERSION": 5,
    "NO_DATA": 6
  },
  "updateRate": 100,
  "ethernetUDPPort": 11000,
  "ethernetTCPPort": 12000,
  "MACaddressPrefix": [222, 173],
  "headerLength": 6,
  "RovecommManifest": { ... }
}
```

3.1.1 Board Declaration Schema

Under **RovecommManifest**, each physical board or virtual simulator is declared as an object containing its static IP address, commands, telemetry, errors, and enums:

```

"Core": {
  "Ip": "192.168.2.110",
  "Commands": {
    "DriveLeftRight": {
      "dataId": 3000,
      "dataType": "FLOAT_T",
      "dataCount": 2,
      "comments": "[LeftSpeed, RightSpeed] (-1 - 1) -> (-100% - 100%)"
    },
    "StateDisplay": {
      "dataId": 3010,
      "dataType": "UINT8_T",
      "dataCount": 1,
      "comments": "[State] (DisplayState)"
    }
  },
  "Telemetry": {
    "MotorSpeeds": {
      "dataId": 3100,
      "dataType": "FLOAT_T",
      "dataCount": 6,
      "comments": "[FL, ML, BL, FR, MR, BR] (-1 - 1) -> (-100% - 100%)"
    }
  },
  "Error": {
    "VESCFault": {
      "dataId": 3200,
      "dataType": "UINT8_T",
      "dataCount": 2,
      "comments": "[MotorID, FaultCode]"
    }
  },
  "enums": {
    "DisplayState": [
      "TELEOP",
      "AUTONOMY",
      "REACHED_GOAL"
    ]
  }
}

```

3.1.2 Comment Formatting Conventions

To allow automated parsers and downstream developers to understand packet semantics instantly, the `comments` string follows a strict convention: "[Param1, Param2, ...] (Unit1, Unit2, ...) Description"

- **Parameter Names:** Wrapped in brackets [...] matching array indices.
- **Units:** Wrapped in parentheses (...), e.g., (deg, m/s, A, V, mm). If a parameter is an enum, the enum name is placed in parentheses (e.g., (DisplayState)). If it is a normalized ratio, the range is specified: (-1 - 1) -> (-100% - 100%).
- **Textual Description:** Explains physical behavior, safety limits, or operational nuances.

3.2 2. Automated README Generation

Inside RoveComm_Base, `.github/workflows/readme-generate.yml` runs whenever a commit is pushed to master:

```
name: Generate README from Manifest
on:
  push:
    branches: ["master"]

permissions:
  contents: write

jobs:
  generate_readme:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Set up Python environment
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Run README generation script
        run: |
          python .github/scripts/readme-generate.py --json ./manifest.json --output ./README.md
```

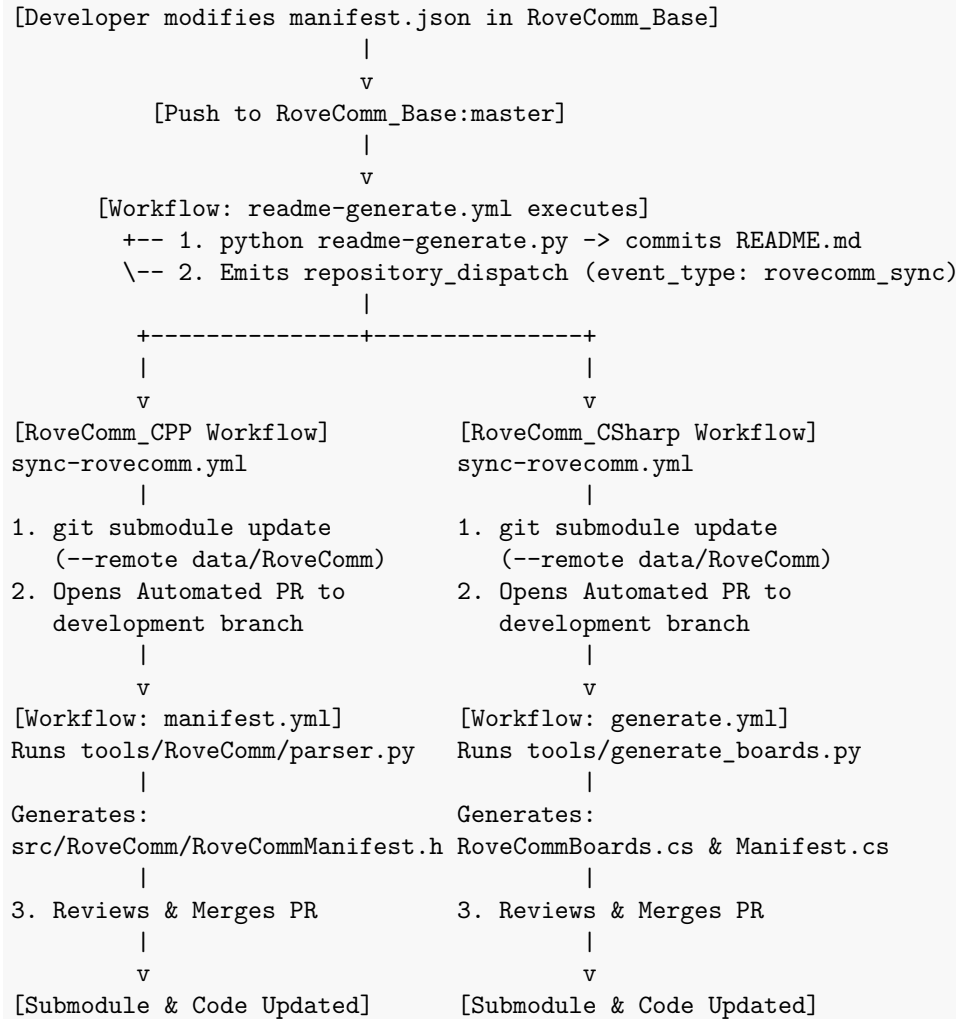
The script `.github/scripts/readme-generate.py` parses `manifest.json`, validates schema structure, and formats every board, command, telemetry metric, and error into GitHub-Flavored Markdown tables. If changes are detected, it commits `README.md` directly back to `master` with the message `[Automated] Human Readable Manifest Update`.

3.3 3. The Webhook Dispatch Pipeline

Immediately after updating `README.md`, the workflow notifies downstream implementations via GitHub's `repository_dispatch` REST API:

```
- name: Update RoveComm_CPP
  run: |
    curl -L \
      -X POST \
      -H "Accept: application/vnd.github+json" \
      -H "Authorization: Bearer ${ secrets.RC_PAT_TOKEN }" \
      -H "X-GitHub-API-Version: 2022-11-28" \
      https://api.github.com/repos/MissouriMRDT/RoveComm_CPP/dispatches \
      -d '{"event_type":"rovecomm_sync", "client_payload":{}}'
```

3.3.1 End-to-End Synchronization Architecture



3.4 4. Downstream Code Binding Generators

3.4.1 C++ Code Generation (RoveComm_CPP)

In `MissouriMRDT/RoveComm_CPP`, the script `tools/RoveComm/parser.py` parses `data/RoveComm/manifest.json` and produces `src/RoveComm/RoveCommManifest.h`.

This header generates type-safe C++ namespaces and data structs:

```

namespace manifest
{
    namespace Core
    {
        const AddressEntry IP = {192, 168, 2, 110};

        namespace Commands
        {
            const ManifestEntry DriveLeftRight = {3000, FLOAT_T, 2};
            const ManifestEntry DriveIndividual = {3001, FLOAT_T, 6};
        }
    }
}

```

```

    const ManifestEntry StateDisplay = {3010, UINT8_T, 1};
}

namespace Telemetry
{
    const ManifestEntry MotorSpeeds = {3100, FLOAT_T, 6};
    const ManifestEntry MotorCurrents = {3101, FLOAT_T, 6};
}

namespace Enums
{
    enum class DisplayState
    {
        TELEOP = 0,
        AUTONOMY = 1,
        REACHED_GOAL = 2
    };
}
}

```

This guarantees compile-time validation: if a developer attempts to send an invalid data ID or mismatching data type in Autonomy Software, the C++ compiler catches it immediately.

3.4.2 C# Code Generation (RoveComm_CSharp)

In MissouriMRDT/RoveComm_CSharp, tools/generate_boards.py and tools/parser.py generate strongly-typed classes:

- **RoveCommBoards.cs**: Static definitions of each board with typed command methods and telemetry observables.
- **RoveCommManifest.cs**: In-memory dictionary enabling dynamic packet lookup by name or **DataID** for the BaseStation UI and diagnostic logs.

3.5 5. How to Add a New Packet to RoveComm (Tutorial)

When integrating a new sensor (such as the Doppler velocity sensor) or actuator into the rover network, follow this standard procedure:

1. Clone RoveComm_Base:

```

git clone https://github.com/MissouriMRDT/RoveComm_Base.git
cd RoveComm_Base
git checkout -b feature/<sensor-name>-packet

```

2. **Edit manifest.json**: Navigate to the appropriate board section (e.g., Nav for navigation sensors). Choose an unused **dataId** in the board's range (e.g., 11100 for Nav telemetry):

```

"GPSVelocity": {
    "dataId": 11105,
    "dataType": "FLOAT_T",
    "dataCount": 3,
    "comments": "[VelX, VelY, VelZ] (m/s) Doppler GPS ground velocity vector in NED frame"
}

```

3. **Verify Locally:** Run the README generator script locally to ensure JSON syntax is valid:

```
python .github/scripts/readme-generate.py --json manifest.json --output README.md
git diff README.md
```

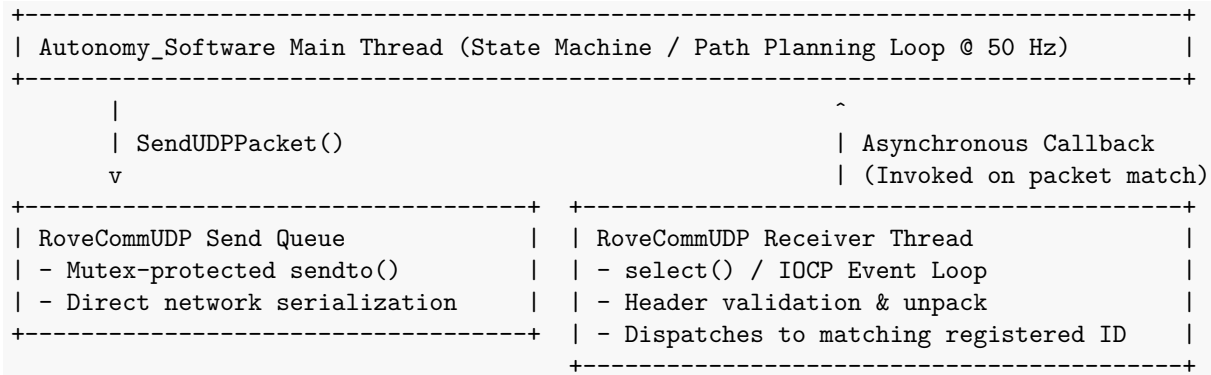
4. **Submit Pull Request:** Commit `manifest.json` and push to GitHub. Open a Pull Request targeting `master`. Once reviewed by software leadership and merged, the automated CI pipeline automatically propagates the new packet to `RoveComm_CPP`, `RoveComm_CSharp`, and all dependent robotics repositories!

4 Chapter 04: C++ Implementation Guide

The C++ implementation of RoveComm (`MissouriMRDT/RoveComm_CPP`) is the high-performance communications engine powering both the onboard rover software stack (`Autonomy_Software`) and the Unreal Engine 5 digital twin simulator (`RoveSoSimulator`). Engineered in modern C++20, it provides asynchronous non-blocking network I/O, multi-threaded worker pools, and compile-time type safety.

4.1 1. Architecture & Threading Model

Network I/O in high-speed robotics must never block the main perception or path-planning control loops. In `RoveComm_CPP`, all socket listening and callback dispatches execute in dedicated background threads managed by `AutonomyThread`:



4.1.1 Platform Compatibility

`RoveComm_CPP` abstracts OS-level socket differences: - **Linux (Jetson AGX Orin / Ubuntu 22.04 / 24.04)**: Standard non-blocking POSIX sockets using `sys/socket.h`, `netinet/in.h`, and `arpa/inet.h`. - **Windows (Development PCs / Unreal Engine 5)**: Windows Sockets 2 (`ws2tcpip.h`, `winsock2.h`) with optional I/O Completion Ports (IOCP) for asynchronous message dispatching.

4.2 2. Core Classes & Data Structures

4.2.1 `rovecomm::RoveCommPacket<T>`

The templated struct representing an in-memory packet. Type parameter `T` must be an arithmetic primitive matching the manifest (`int8_t`, `uint8_t`, `int16_t`, `uint16_t`, `int32_t`, `uint32_t`, `float`, `double`, or `char`):

```
template<typename T>
struct RoveCommPacket
{
    uint16_t unDataId;
    uint16_t unDataCount;
    manifest::DataTypes eDataType;
    std::vector<T> vData;
};
```

4.2.2 `rovecomm::RoveCommUDP`

Manages UDP transmission, subscription tracking, and asynchronous datagram reception:

```

class RoveCommUDP : AutonomyThread<void>
{
public:
    RoveCommUDP();
    ~RoveCommUDP();

    bool InitUDPSocket(int nPort);

    template<typename T>
    ssize_t SendUDPPacket(const RoveCommPacket<T>& stPacket, const char* cIPAddress, int nPort);

    template<typename T>
    void AddUDPCallback(std::function<void(const RoveCommPacket<T>&, const sockaddr_in&)> fnCallback,
                        const uint16_t& unCondition);

    template<typename T>
    void RemoveUDPCallback(std::function<void(const RoveCommPacket<T>&, const sockaddr_in&)> fnCallback);
};

```

4.2.3 rovecomm::RoveCommTCP

Manages connection-oriented TCP socket streams, boundary reassembly, and reliable command transmission:

```

class RoveCommTCP : AutonomyThread<void>
{
public:
    RoveCommTCP();
    ~RoveCommTCP();

    bool InitTCPSocket(const char* cIPAddress, int nPort);

    template<typename T>
    ssize_t SendTCPPacket(const RoveCommPacket<T>& stData, const char* cClientIPAddress, int nClientPort);

    template<typename T>
    void AddTCPCallback(std::function<void(const RoveCommPacket<T>&)> fnCallback,
                        const uint16_t& unCondition);

    void CloseTCPSocket();
};

```

4.3 3. Production Code Examples

4.3.1 Example 1: Transmitting Drive Commands over UDP

This pattern demonstrates how the Autonomy control loop sends differential wheel speeds to the Core Board:

```

#include "RoveComm/RoveComm.h"
#include "RoveComm/RoveCommManifest.h"

void DriveRover(rovecomm::RoveCommUDP& udpNode, float leftSpeed, float rightSpeed)
{
    // 1. Construct RoveComm packet using manifest metadata
    rovecomm::RoveCommPacket<float> drivePacket;

```

```

drivePacket.unDataId = manifest::Core::Commands::DriveLeftRight.DATA_ID;
drivePacket.eDataType = manifest::Core::Commands::DriveLeftRight.DATA_TYPE;
drivePacket.unDataCount = manifest::Core::Commands::DriveLeftRight.DATA_COUNT; // 2
drivePacket.vData = {leftSpeed, rightSpeed};

// 2. Transmit to Core Board IP on default UDP port 11000
const char* coreIP = manifest::Core::IP.IP_STR.c_str();
ssize_t bytesSent = udpNode.SendUDPPacket(drivePacket, coreIP, ROVECOMM_ETHERNET_UDP_PORT);

if (bytesSent < 0)
{
    // Handle socket transmission error
}
}

```

4.3.2 Example 2: Subscribing to GPS Telemetry with Asynchronous Callbacks

This pattern demonstrates how an autonomy sensor driver subscribes to Nav Board telemetry and processes GPS updates asynchronously:

```

#include "RoveComm/RoveComm.h"
#include "RoveComm/RoveCommManifest.h"
#include <iostream>

class GPSReceiver
{
private:
    rovecomm::RoveCommUDP m_udpNode;

    void OnGPSPoseReceived(const rovecomm::RoveCommPacket<double>& packet, const sockaddr_in& senderAddr)
    {
        if (packet.vData.size() >= 3)
        {
            double latitude = packet.vData[0];
            double longitude = packet.vData[1];
            double altitude = packet.vData[2];
            std::cout << "Received GPS Pose: Lat=" << latitude << ", Lon=" << longitude << "\n";
        }
    }

public:
    bool Initialize()
    {
        // 1. Bind local UDP listening socket
        if (!m_udpNode.InitUDPSocket(ROVECOMM_ETHERNET_UDP_PORT))
        {
            return false;
        }

        // 2. Register callback matching NavBoard GPSPose DataID (11100)
        m_udpNode.AddUDPCallback<double>(
            [this](const rovecomm::RoveCommPacket<double>& pkt, const sockaddr_in& addr) {
                this->OnGPSPoseReceived(pkt, addr);
            },

```

```

        manifest::Nav::Telemetry::GPS Pose.DATA_ID
    );

    // 3. Send SUBSCRIBE packet to NavBoard
    rovecomm::RoveCommPacket<uint8_t> subPacket;
    subPacket.unDataId = manifest::SystemPackets::SUBSCRIBE;
    subPacket.eDataType = manifest::DataTypes::UINT8_T;
    subPacket.unDataCount = 0;
    subPacket.vData = {};

    m_udpNode.SendUDPPacket(subPacket, manifest::Nav::IP.IP_STR.c_str(), ROVECOMM_ETHERNET_UDP_PORT);
    return true;
}
};

```

4.3.3 Example 3: Transmitting Critical E-Stop Command over TCP

```

#include "RoveComm/RoveComm.h"
#include "RoveComm/RoveCommManifest.h"

bool TriggerEmergencyStop(rovecomm::RoveCommTCP& tcpNode)
{
    // EStop packet on PMS Board (DataID: 4000, DataCount: 0)
    rovecomm::RoveCommPacket<uint8_t> estopPacket;
    estopPacket.unDataId = manifest::PMS::Commands::EStop.DATA_ID;
    estopPacket.eDataType = manifest::PMS::Commands::EStop.DATA_TYPE;
    estopPacket.unDataCount = 0;
    estopPacket.vData = {};

    ssize_t bytes = tcpNode.SendTCPPacket(
        estopPacket,
        manifest::PMS::IP.IP_STR.c_str(),
        ROVECOMM_ETHERNET_TCP_PORT
    );

    return (bytes > 0);
}

```

4.4 4. CMake Integration in Downstream Projects

To use RoveComm_CPP in any downstream C++ repository (such as Autonomy_Software or RoveSoSimulator):

```

# Add RoveComm as submodule or subdirectory
add_subdirectory(external/rovecomm)

# Link to application target
target_link_libraries(Autonomy_Software PRIVATE RoveComm_CPP)
target_include_directories(Autonomy_Software PRIVATE external/rovecomm/src)

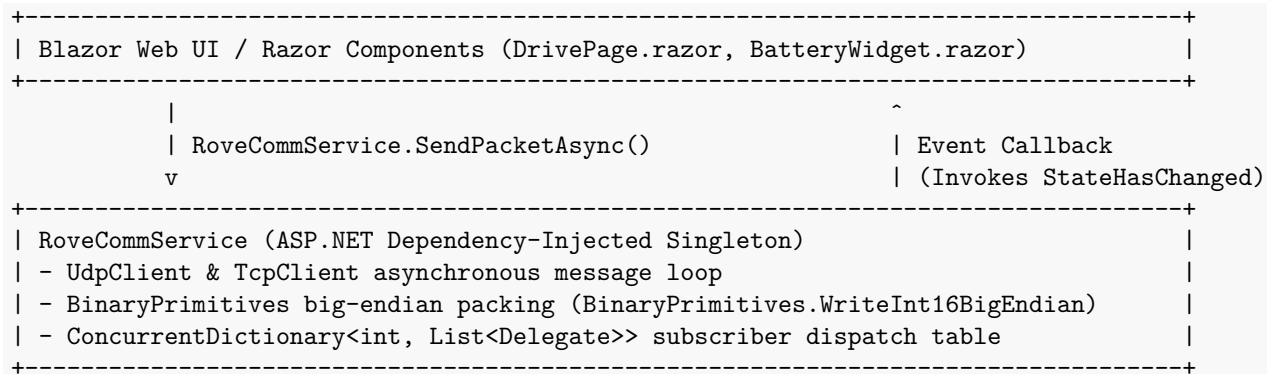
```

5 Chapter 05: C# (.NET / Blazor) Implementation Guide

The C# implementation of RoveComm (MissouriMRDT/RoveComm_CSharp) is the official networking layer for MRDT's mission control software (Basestation_Software_Blazor). Hosted publicly on [NuGet.org](https://www.nuget.org/packages/RoveComm_CSharp), it integrates natively with modern .NET 8 / 9 asynchronous paradigms, dependency injection, and Blazor web components.

5.1 1. Architecture & BaseStation Integration

Modern BaseStation runs as a high-performance Blazor web application. RoveComm_CSharp is engineered as a thread-safe singleton service (RoveCommService) that runs in the background of the ASP.NET runtime, handling non-blocking socket reads via `async/await` and dispatching telemetry updates to web UI components via C# delegates:



5.2 2. Core API Reference

5.2.1 RoveCommPacket<T>

Represents a strongly-typed packet. Valid generic types: `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `float`, `double`, or `char`:

```
namespace RoveComm;

public class RoveCommPacket<T>
{
    public int DataID { get; set; }
    public int DataCount => Data.Count;
    public RoveCommDataType DataType { get; init; }
    public List<T> Data { get; set; }

    public RoveCommPacket(int dataId, IEnumerable<T> data);
    public RoveCommPacket(int dataId, int dataCount);
    public RoveCommHeader GetHeader();
}
```

5.2.2 RoveCommService

The central manager for network sockets and subscription delegates:

```

public class RoveCommService
{
    // Transmit packets
    public async Task<int> SendPacket<T>(RoveCommPacket<T> packet, string ip, int port = 11000);

    // Register callbacks for incoming packets
    public void Subscribe<T>(int dataId, RoveCommCallback<T> callback);
    public void Unsubscribe<T>(int dataId, RoveCommCallback<T> callback);

    // TCP client connection management
    public async Task ConnectTCP(string ip, int port = 12000);
}

```

5.3 3. Production Code Examples

5.3.1 Example 1: Registering RoveCommService in Program.cs

In a Blazor or ASP.NET application, register RoveComm as a singleton service:

```

using RoveComm;

var builder = WebApplication.CreateBuilder(args);

// Add Blazor services
builder.Services.AddRazorPages();
builder.Services.AddServerSideBlazor();

// Register RoveComm as a Singleton
builder.Services.AddSingleton<RoveCommService>();

var app = builder.Build();
app.Run();

```

5.3.2 Example 2: Subscribing to Telemetry in a Blazor Component

This Razor component binds real-time battery voltage and current from the PMS Board (DataID = 4100) directly to a UI widget:

```

@page "/pms-monitor"
@using RoveComm
@inject RoveCommService RoveComm
@implements IDisposable

<h3>Power Management System Telemetry</h3>

@if (batteryData != null)
{
    <p><strong>Pack Current:</strong> @batteryData[0] A</p>
    <p><strong>Aux Current:</strong> @batteryData[1] A</p>
    <p><strong>Cell 1 Voltage:</strong> @batteryData[6] V</p>
    <p><strong>Cell 2 Voltage:</strong> @batteryData[7] V</p>
}
else
{

```

```

    <p><em>Waiting for PMS telemetry stream...</em></p>
}

@code {
    private List<float>? batteryData;

    protected override async Task OnInitializedAsync()
    {
        // 1. Register callback for PMS CurrentAndVoltage packet (ID: 4100)
        RoveComm.Subscribe<float>(4100, OnPMSDataReceived);

        // 2. Transmit SUBSCRIBE request to PMS Board IP (192.168.2.102)
        var subPacket = new RoveCommPacket<byte>(3, Array.Empty<byte>());
        await RoveComm.SendPacket(subPacket, "192.168.2.102", 11000);
    }

    private Task OnPMSDataReceived(RoveCommPacket<float> packet)
    {
        batteryData = packet.Data;
        InvokeAsync(StateHasChanged);
        return Task.CompletedTask;
    }

    public void Dispose()
    {
        // Clean up callback when component unmounts
        RoveComm.Unsubscribe<float>(4100, OnPMSDataReceived);
    }
}

```

5.3.3 Example 3: Dispatching Drive Commands from Joystick Input

```

public async Task UpdateDriveSpeeds(float leftPower, float rightPower)
{
    // Clamp speeds to valid normalized range (-1.0 to 1.0)
    leftPower = Math.Clamp(leftPower, -1.0f, 1.0f);
    rightPower = Math.Clamp(rightPower, -1.0f, 1.0f);

    // Core Board DriveLeftRight command (DataID: 3000, 2 floats)
    var packet = new RoveCommPacket<float>(3000, new float[] { leftPower, rightPower });

    // Send UDP packet to Core Board IP
    await _roveCommService.SendPacket(packet, "192.168.2.110", 11000);
}

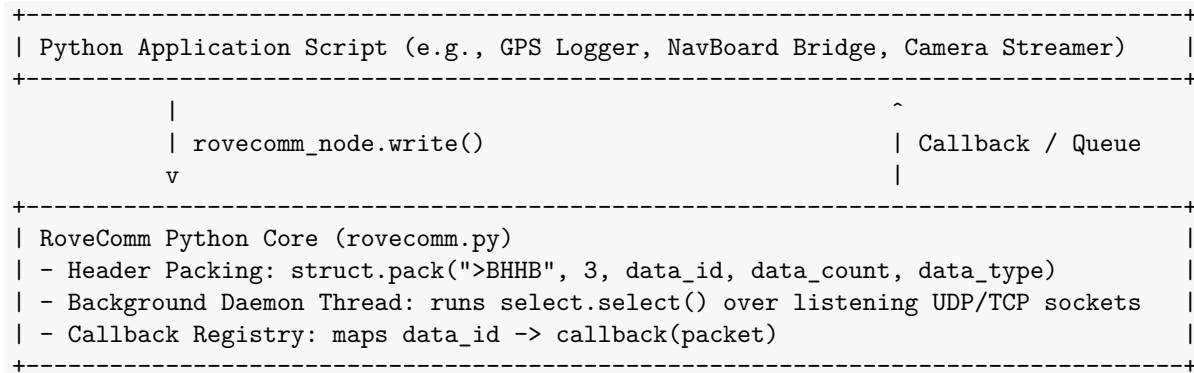
```

6 Chapter 06: Python Implementation Guide

The Python implementation of RoveComm (MissouriMRDT/RoveComm_Python and RoveComm_Tester_Software/RoveComm_Te) is widely used across MRDT for auxiliary robotics services: - Differential GPS correction injection on the Navigation Board (NavBoard). - Video feed control and status telemetry on Raspberry Pi camera nodes. - Automated hardware-in-the-loop diagnostic scripts and regression test suites. - The desktop diagnostic GUI (RoveComm_Tester_Software).

6.1 1. Architecture of rovecomm.py

The Python implementation encapsulates packet packing and network transmission using standard library modules (socket, struct, threading, select):



6.1.1 Struct Packing & Header Format

In Python, binary conversion is performed with the format string ">BHBB": - >: Network byte order (Big-Endian). - B: 1-byte unsigned char (Version = 3). - H: 2-byte unsigned short (DataID). - H: 2-byte unsigned short (DataCount). - B: 1-byte unsigned char (DataType).

```
ROVECOMM_HEADER_FORMAT = ">BHBB"
ROVECOMM_VERSION = 3
```

6.2 2. Core Python Classes

6.2.1 RoveCommPacket

```
class RoveCommPacket:
    def __init__(self, data_id=0, data_type="b", data=(), ip="", port=11000):
        self.data_id = data_id
        self.data_type = data_type
        self.data_count = len(data)
        self.data = data
        self.ip_address = (ip, port) if ip else ("0.0.0.0", 0)
```

6.2.2 RoveComm Node

```
class RoveComm:
    def __init__(self, udp_port=11000, tcp_port=12000):
        # Initializes non-blocking sockets and launches background listening daemon
```

```

...

def write(self, packet: RoveCommPacket, use_tcp=False):
    # Packs header and payload, transmits over UDP or TCP
    ...

def set_callback(self, data_id: int, callback_fn):
    # Registers callback function triggered when packet with data_id arrives
    ...

```

6.3 3. Production Code Examples

6.3.1 Example 1: Publishing GPS Doppler Velocity from Python

This script illustrates how the NavBoard Python bridge reads Doppler ground speed from an external GNSS receiver and publishes it over RoveComm for Autonomy Software:

```

import time
from rovecomm_module.rovecomm import RoveComm, RoveCommPacket

# Initialize RoveComm Node
rc = RoveComm()

# Destination: Autonomy Jetson Computer
AUTONOMY_IP = "192.168.2.108"
AUTONOMY_PORT = 11000

# NavBoard GPSVelocity telemetry (DataID: 11105, 3 floats)
# Data: [VelocityNorth, VelocityEast, VelocityDown] in m/s
def publish_velocity(vel_north, vel_east, vel_down):
    packet = RoveCommPacket(
        data_id=11105,
        data_type="f", # 4-byte IEEE 754 float
        data=(vel_north, vel_east, vel_down),
        ip=AUTONOMY_IP,
        port=AUTONOMY_PORT
    )
    rc.write(packet, use_tcp=False)

# Main publication loop @ 20 Hz
while True:
    # Simulated 1.2 m/s forward velocity
    publish_velocity(1.2, 0.0, 0.0)
    time.sleep(0.05)

```

6.3.2 Example 2: Subscribing & Logging Rover Motor Currents

This script subscribes to Core Board motor currents and logs them to a CSV file for post-run energy analysis:

```

import csv
import time
from rovecomm_module.rovecomm import RoveComm, RoveCommPacket

rc = RoveComm()

```

```

CORE_BOARD_IP = "192.168.2.110"

# Open CSV log file
csv_file = open("motor_currents.csv", "w", newline="")
writer = csv.writer(csv_file)
writer.writerow(["Timestamp", "FL", "ML", "BL", "FR", "MR", "BR"])

def on_motor_currents_received(packet: RoveCommPacket):
    if len(packet.data) == 6:
        timestamp = time.time()
        writer.writerow([timestamp] + list(packet.data))
        csv_file.flush()

# 1. Register callback for Core Board MotorCurrents (ID: 3101)
rc.set_callback(3101, on_motor_currents_received)

# 2. Subscribe to Core Board
sub_packet = RoveCommPacket(
    data_id=3,          # SUBSCRIBE
    data_type="B",
    data=(),
    ip=CORE_BOARD_IP,
    port=11000
)
rc.write(sub_packet)

print("Subscribed to Core Board. Logging motor currents... Press Ctrl+C to exit.")
try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    csv_file.close()
    print("Logging complete.")

```

7 Chapter 07: Embedded (Arduino / Microcontroller) Guide

The embedded implementation of RoveComm (`MissouriMRDT/RoveComm_Arduino`) operates on the physical microcontroller boards distributed across the rover chassis. These boards – primarily **Teensy 4.1** (ARM Cortex-M7 running at 600 MHz) and **STM32** controllers running the Arduino framework – directly actuate drive motors, monitor battery cells, control robotic arm joints, and trigger science mechanisms.

7.1 1. Embedded Constraints & Architecture

Microcontroller environments impose strict constraints that differ radically from desktop operating systems:

1. **Zero Dynamic Allocation:** Calling `malloc()`, `free()`, or constructing dynamic `std::vector` objects within high-frequency control loops introduces memory fragmentation and non-deterministic latency. All RoveComm buffers are statically allocated at compile time.
2. **Polling vs Multi-Threading:** Most microcontrollers run bare-metal or on lightweight RTOSs without operating-system thread pools. RoveComm processing executes inside the main `loop()` through non-blocking `read()` checks.
3. **Hardware Ethernet PHY:** Teensy 4.1 utilizes the onboard 10/100 Mbit DP83825I PHY transceiver driven by the `NativeEthernet` library, providing direct hardware MAC and IP stack acceleration.

```
+-----+
| Teensy 4.1 Microcontroller Main Loop (100 Hz Cycle) |
+-----+
|
|   | rovecomm.read() [Non-blocking]
|   v
+-----+
| Check for Incoming Packets |
| - Parse 6-byte header      |
| - Switch on DataID:        |
| * SUBSCRIBE   -> Add to static subscriber list |
| * UNSUBSCRIBE -> Remove from subscriber list  |
| * Command ID  -> Update Actuator PWM/CAN state |
+-----+
|
|   v
+-----+
| Telemetry Transmission (Periodic @ 20 Hz) |
| - Sample ADC / CAN bus sensor data         |
| - Iterate through static subscriber table   |
| - Transmit unicast UDP packets              |
+-----+
```

7.2 2. Production Firmware Example

The following code illustrates a production-ready microcontroller firmware loop implementing the Core Board drive receiver:

```
#include <NativeEthernet.h>
#include <NativeEthernetUdp.h>
#include <RoveComm.h>

// Static IP and Network Configuration from manifest.json
```

```

IPAddress boardIP(192, 168, 2, 110);
IPAddress gateway(192, 168, 2, 1);
IPAddress subnet(255, 255, 0);
byte macAddress[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0x02, 0x6E };

// RoveComm Ethernet UDP Instance
EthernetUDP udpSocket;
RoveCommEthernetUdp rovecomm(udpSocket);

// Actuator State
float targetLeftSpeed = 0.0f;
float targetRightSpeed = 0.0f;
unsigned long lastCommandTime = 0;
const unsigned long WATCHDOG_TIMEOUT_MS = 500;

void setup()
{
    Serial.begin(115200);
    Ethernet.begin(macAddress, boardIP, gateway, gateway, subnet);
    rovecomm.begin(ROVECOMM_ETHERNET_UDP_PORT);
    Serial.println("Core Board RoveComm Node Initialized.");
}

void loop()
{
    // 1. Process Incoming Packets
    rovecomm_packet packet;
    if (rovecomm.read(&packet))
    {
        switch (packet.data_id)
        {
            // Core Board DriveLeftRight Command (DataID: 3000)
            case 3000:
                if (packet.data_type == ROVECOMM_FLOAT_T && packet.data_count == 2)
                {
                    float* speeds = (float*)packet.data;
                    targetLeftSpeed = speeds[0];
                    targetRightSpeed = speeds[1];
                    lastCommandTime = millis();
                }
                break;

            // Power Management System E-Stop
            case 4000:
                targetLeftSpeed = 0.0f;
                targetRightSpeed = 0.0f;
                break;
        }
    }

    // 2. Hardware Watchdog Timeout (Safety Invariant)
    if (millis() - lastCommandTime > WATCHDOG_TIMEOUT_MS)
    {

```

```

    targetLeftSpeed = 0.0f;
    targetRightSpeed = 0.0f;
}

// 3. Command Motor Actuators via CAN Bus
ApplyMotorPowers(targetLeftSpeed, targetRightSpeed);

// 4. Stream Telemetry to Subscribers @ 20 Hz
static unsigned long lastTelemetryTime = 0;
if (millis() - lastTelemetryTime >= 50)
{
    lastTelemetryTime = millis();
    float currentWheelSpeeds[6] = { targetLeftSpeed, targetLeftSpeed, targetLeftSpeed,
                                     targetRightSpeed, targetRightSpeed, targetRightSpeed };

    // Transmits MotorSpeeds (DataID: 3100) to all registered subscribers
    rovecomm.write(3100, ROVECOMM_FLOAT_T, 6, currentWheelSpeeds);
}
}

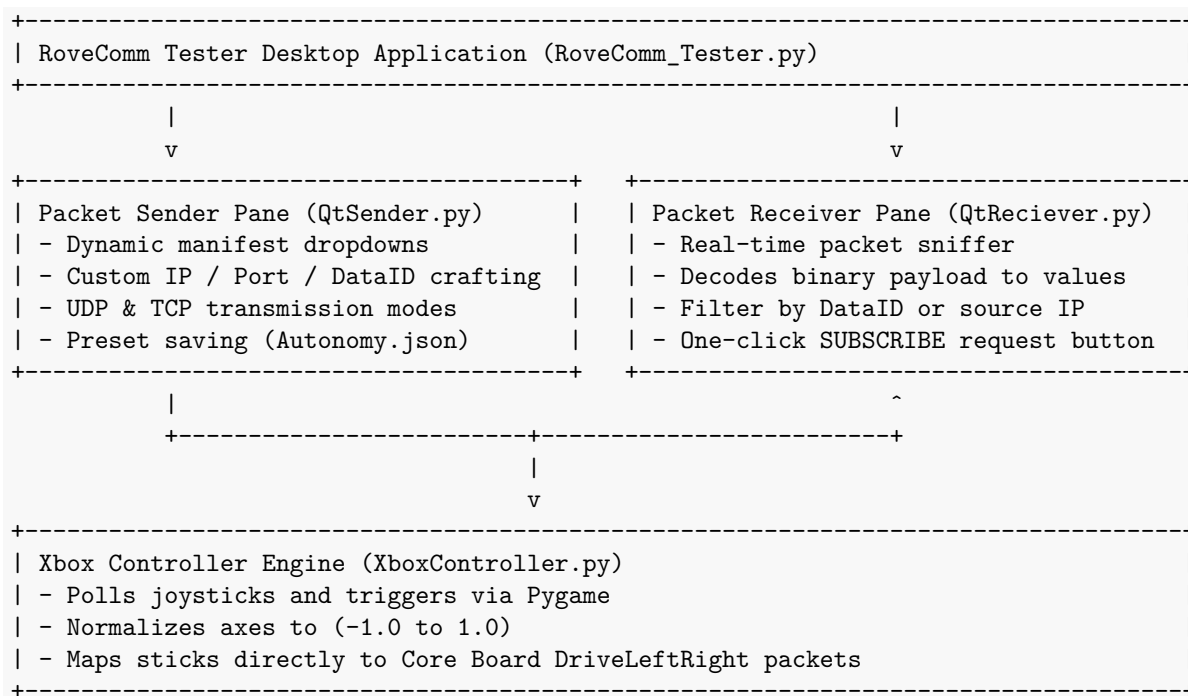
```

8 Chapter 08: RoveComm Tester Software & Diagnostic Tooling

RoveComm_Tester_Software (MissouriMRDT/RoveComm_Tester_Software) is MRDT's primary graphical diagnostics and hardware-in-the-loop validation utility. Written in Python 3 using PyQt5, it enables software and electrical engineers to inspect raw packet streams, craft and inject arbitrary command packets, simulate missing rover subsystems during software development, and teleoperate rover mechanisms directly using an Xbox gamepad without running the full BaseStation suite.

8.1 1. Overview & System Architecture

RoveComm_Tester_Software couples a dual-pane PyQt5 interface with the underlying rovecomm.py networking engine:



8.2 2. Installation & Quick Start

8.2.1 Prerequisites

- Python 3.10+
- Pip package manager

8.2.2 Setup Instructions

```
# Clone the repository
git clone --recurse-submodules https://github.com/MissouriMRDT/RoveComm_Tester_Software.git
cd RoveComm_Tester_Software

# Install dependencies
pip install -r requirements.txt
pip install PyQt5 pygame
```

```
# Launch the Tester GUI
python RoveComm_Tester/RoveComm_Tester.py
```

8.3 3. Core Modules Deep Dive

8.3.1 A. The Packet Sender (QtSender.py)

The sender module allows interactive crafting of any packet registered in `manifest.json`:

1. **Board & Packet Dropdown:** Select the target board (e.g., Core, PMS, Nav) and command/telemetry name. The tool automatically populates the destination IP, port, default Data ID, and data type.
2. **Payload Editor:** Dynamically renders input fields based on `dataCount` and `dataType`. Array elements are validated against valid integer or float ranges before transmission.
3. **Transport Toggle:** Switch seamlessly between connectionless UDP (Port 11000) and reliable TCP (Port 12000).
4. **Configuration Presets:** Save frequently used test sequences (e.g., 1-Configs/Autonomy.json, DriveConfig.json) to disk and recall them with a single click.

8.3.2 B. The Packet Sniffer & Receiver (QtReceiver.py)

The receiver module binds to local UDP port 11000 and TCP port 12000:

1. **Live Feed Table:** Displays timestamp, source IP, Data ID, decoded packet name (resolved from manifest), data count, and unpacked values.
2. **Filtering:** Filter incoming traffic by specific `DataID` (e.g., isolate 3100 `MotorSpeeds`) or source IP.
3. **Auto-Subscribe:** Selecting a board and clicking **Subscribe** emits an automated SUBSCRIBE packet (`DataID = 3`), prompting the board to begin streaming telemetry to the tester PC.

8.3.3 C. Gamepad Teleoperation (XboxController.py)

For rapid chassis testing without deploying BaseStation, `RoveComm_Tester` interfaces directly with Xbox One and Series X controllers via `pygame.joystick`:

- **Left Stick Vertical:** Controls left wheel bank speed (−1.0 to 1.0).
 - **Right Stick Vertical:** Controls right wheel bank speed (−1.0 to 1.0).
 - **Right Bumper (Deadman Switch):** Must be depressed for drive packets to transmit, preventing accidental rover runaway.
 - **D-Pad:** Commands camera gimbals on the Multimedia Board (`LeftGimbal`, `RightGimbal`).
-

8.4 4. Common Diagnostic Workflows

8.4.1 Diagnostic 1: Verifying Board Network Liveness (Ping)

When an embedded board fails to respond to commands:

1. Launch `RoveComm_Tester`.
2. Set Target IP to the board's static IP (e.g., 192.168.2.110 for Core).
3. Set Data ID to 1 (PING), Data Type to `UINT8_T`, Data Count to 0.
4. Click **Send UDP**.
5. Observe the Receiver table: a healthy board responds within 10 ms with `DataID = 2` (PING_REPLY). If no reply is received, investigate Ethernet physical layer (link lights on the switch) or static IP configuration.

8.4.2 Diagnostic 2: Simulating Sensor Telemetry for Autonomy Testing

When developing Autonomy navigation algorithms without physical rover hardware:

1. Launch **RoveComm_Tester** on a secondary computer (or local loopback).
2. Configure Sender to transmit Nav Board GPSPose (DataID = 11100, Data Type: DOUBLE_T, Count: 3).
3. Enter current test coordinates: [37.95155, -91.77812, 320.0].
4. Enable **Continuous Send @ 10 Hz**.
5. In **Autonomy_Software**, verify that the State Machine detects valid GPS lock and transitions from IDLE to NAVIGATING.

9 Chapter 09: RoveSoDocs Hub Integration Playbook

This chapter outlines the exact integration procedure that connects the **RoveComm Protocol Guide** to MRDT's centralized documentation hub at docs.themrtdt.org.

By following the dual-docset architecture pioneered by the Autonomy Software documentation team (Chapter 16 of the Autonomy Binder), the high-level **RoveComm Protocol Guide (Jekyll)** is deployed alongside the low-level **RoveComm C++ API Reference (Doxygen)** without path conflicts or routing collisions.

9.1 1. Dual-Docset Subpath Convention

On `docs.themrtdt.org`, the `/rovecomm/` URL space is cleanly partitioned into two complementary docsets:

Route Path	Generator / Engine	Source Repository & Branch	Purpose
<code>/rovecomm/_j/</code>	Jekyll 3.10	MissouriMRDT/RoveComm_Base (docs)	Protocol Architecture & Guide: Conceptual manual, wire format, manifest ecosystem, multi-language guides, and tester tooling.
<code>/rovecomm/_cpp/</code>	Doxygen	MissouriMRDT/RoveComm_CPP-dev	C++ API Reference: Low-level class inheritance, header documentation, struct definitions, and Doxygen call graphs.

9.2 2. The 6-Step Integration Checklist

To wire `RoveComm_Base` into the central documentation pipeline:

9.2.1 Step 1: Duplicate Jekyll Section in RoveSoDocs/.github/workflows/deploy.yml

In `MissouriMRDT/RoveSoDocs`, add a dedicated build job named `build_rovecomm_jekyll`:

```
# Build RoveComm Protocol Guide (Jekyll)
build_rovecomm_jekyll:
  runs-on: ubuntu-latest
  steps:
    - name: Checkout RoveComm Base (Docs Branch)
      uses: actions/checkout@v4
      with:
        repository: MissouriMRDT/RoveComm_Base
        ref: docs/rovecomm
        path: src/RoveComm_Base

    - name: Set up Ruby
      uses: ruby/setup-ruby@v1
      with:
        ruby-version: "3.1"
```

```

    bundler-cache: true
    working-directory: src/RoveComm_Base/docs

- name: Build Jekyll Site
  run: |
    bundle exec jekyll build --baseurl "/rovecomm/_j"
    working-directory: src/RoveComm_Base/docs

- name: Stage RoveComm Guide into out/
  run: |
    mkdir -p out/rovecomm/_j
    rsync -a src/RoveComm_Base/docs/_site/ out/rovecomm/_j/

- name: Upload RoveComm Jekyll Artifact
  uses: actions/upload-artifact@v4
  with:
    name: rovecomm_jekyll_site
    path: out/rovecomm/_j/
    retention-days: 1

```

9.2.2 Step 2: Merge Artifact in assemble Job

Add `build_rovecomm_jekyll` to the `needs:` array of the `assemble` job and download the artifact into `dist/rovecomm/_j/`:

```

assemble:
  needs:
    - build_vitepress
    - build_autonomy_jekyll
    - build_autonomy_doxygen
    - build_rovecomm_jekyll
    - build_rovecommcpp_doxygen
    - ...

```

9.2.3 Step 3: Register SPA Fallback in NotFoundContent.vue

VitePress is a Single Page Application (SPA). To prevent client-side routing from intercepting direct hits or page reloads within `/rovecomm/_j/`:

In `.vitepress/theme/NotFoundContent.vue`, add `/rovecomm/_j/` to `refreshPrefixes`:

```

const refreshPrefixes = [
  "/autonomy/_d/",
  "/autonomy/_j/",
  "/embedded/",
  "/rovecomm/_cpp/",
  "/rovecomm/_j/",
  "/RoveSoSimulator/_j/",
];

```

And add a quicklink to the 404 navigation recovery bar:

```

<a href="/rovecomm/_j/" class="quicklink">RoveComm Guide</a>

```

9.2.4 Step 4: Index Guide in MiniSearch Search Engine

In `tools/build-minisearch-index.mjs`, update `sectionFromUrl()` to index every HTML page in `/rovecomm/_j/` under the search facet "RoveComm Protocol Guide":

```
function sectionFromUrl(url) {
  if (url.startsWith("/autonomy/_d/")) return "Autonomy Software (Doxygen)";
  if (url.startsWith("/autonomy/_j/")) return "Autonomy Software (Binder)";
  if (url.startsWith("/rovecomm/_cpp/")) return "RoveComm C++ (Doxygen)";
  if (url.startsWith("/rovecomm/_j/")) return "RoveComm Protocol Guide";
  ...
}
```

9.2.5 Step 5: Add Feature Card on Homepage (`index.md`)

In `RoveSoDocs/index.md`, add a feature card under `features::`:

```
- icon: ":satellite:"
  title: "RoveComm Protocol Guide"
  details: "Universal rover communications manual -- wire format specifications, manifest ecosystem, ..."
  link: /rovecomm/_j/
  linkText: "Open RoveComm Guide"
```

9.2.6 Step 6: Maintain Visual Grid Balance

Ensure the total number of feature cards on the homepage is a multiple of 3 (e.g., 6 or 9 cards) so that the 3-column desktop layout (@media (min-width: 1400px)) remains visually symmetrical.

10 Chapter 10: Pandoc Single Markdown & PDF Compilation

In addition to serving as a responsive web manual on `docs.themrtd.org`, the RoveComm documentation can be compiled into a single, unified, publication-grade PDF manual: `RoveComm_Manual.pdf`.

This enables team members, competition judges at the University Rover Challenge (URC), and new recruits to read the complete protocol manual offline or print physical copies.

10.1 1. Toolchain Prerequisites

Compiling the PDF requires `pandoc` and a LaTeX distribution (`pdflatex`):

10.1.1 Ubuntu / Debian / WSL2

```
sudo apt-get update
sudo apt-get install -y pandoc texlive-latex-base texlive-fonts-recommended texlive-extra-utils texlive
```

10.1.2 Windows (Native)

- Install [Pandoc](#) via Chocolatey or Windows Installer: `choco install pandoc`
 - Install [MiKTeX](#) or [TeX Live](#).
-

10.2 2. Compilation Script (`compile_rovecomm_pandoc.sh`)

MRDT provides an automated compiler script in `tools/compile_rovecomm_pandoc.sh`:

```
# Execute from the root of RoveComm_Base repository
bash tools/compile_rovecomm_pandoc.sh
```

10.2.1 Script Execution Pipeline

1. **Table of Contents Parsing:** The script scans `docs/00_Table_of_Contents.md` to extract chapter file paths in sequential order.
 2. **Markdown Assembly:** It prepends Pandoc YAML metadata (title, author, date, margin, link coloring) and concatenates every chapter file into an intermediate monolith: `docs/RoveComm_Guide_Pandoc.md`.
 3. **LaTeX Page Delimitation:** A `\newpage` macro is inserted between consecutive chapters to ensure each chapter begins on a fresh page.
 4. **Pandoc PDF Generation:** Invokes `pandoc` with `--pdf-engine=pdflatex`, generating a hyperlinked table of contents, numbered section headers, and syntax-highlighted code blocks.
 5. **Output:** Produces `docs/RoveComm_Manual.pdf`.
-

10.3 3. Formatting Rules for Pandoc Compatibility

To guarantee clean PDF compilation without LaTeX errors:

- **Strict ASCII Trees:** Directory structures and flowcharts must use ASCII characters (`|`, `+`, `\`, `l`) rather than Unicode box-drawing symbols (`+-`, `l`, `\`), as standard `pdflatex` fails on multi-byte unicode box characters.
- **Leading Blank Lines:** All bulleted lists and numbered steps must be preceded by a blank line.
- **Backtick Shielding:** Shell variables containing dollar signs (`$VAR`, `$HOME`) must be enclosed in code backticks to prevent MathJax / LaTeX from interpreting them as inline math.